Unit 1 QB

1. Prove by contradiction that "The $\sqrt{2}$ is irrational". 5 Marks

we start with an assumption that is contrary to what we are actually required to prove. Then, using a series of logical deductions from this assumption, we reach an inconsistency – a mathematical or logical error – which enables us to conclude that our original assumption was incorrect.

In this case, we start by supposing that √2 is a rational number. Thus, there will exist integers p and q (where q is non-zero) such that p/q = √2. We also make the assumption that p and q have no common factors. As even if they have common factors we would cancel them to write it in the simplest form. So, let us assume that p and q are co-prime numbers here having no common factor other than 1.

Now, squaring both sides, we have p2/q2 = 2, which can be rewritten as,

p2 = 2{q2}.....(1)

We note that the right-hand side of the equation is multiplied by 2, which means that the left-hand side is a multiple of 2. So, we can say that p2 is a multiple of 2. This further means that p itself must be a multiple of 2, as when a prime number is a factor of a number, let's say, m2, it is also a factor of m. Thus, we can assume that,

p=2m, m∈Z [Set of Integers]

⇒(2m)2= 2q2 [From (1)]

⇒4m2=2q2

⇒q2=2m2

Now, the right-hand side is a multiple of 2 again, which means that the left-hand side is a multiple of 2, which further means that q is a multiple of 2, i.e., q = 2n, where n ∈ Z. We have thus shown that both p and q are multiples of 2. But is that possible? This can only mean one thing: our original assumption of assuming √2 as p/q (where p and q are co-prime integers) is wrong:

√2 ≠ p/q

Thus, √2 does not have a rational representation – √2 is irrational.

2. Compare a priori and posterior analysis of Algorithm. 5 Marks

A Posteriori analysis	A priori analysis
Posteriori analysis is a relative analysis.	Priori analysis is an absolute analysis.
It is dependent on language of compiler and type of hardware.	It is independent of language of compiler and types of hardware.
It will give exact answer.	It will give approximate answer.
It doesn't use asymptotic notations to represent the time complexity of an algorithm.	It uses the asymptotic notations to represent how much time the algorithm will take in order to complete its execution.
The time complexity of an algorithm using a posteriori analysis differ from system to system.	The time complexity of an algorithm using a priori analysis is same for every system.
If the time taken by the program is less, then the credit will go to compiler and hardware.	If the algorithm running faster, credit goes to the programmer.
It is done after execution of an algorithm.	It is done before execution of an algorithm.
It is costlier than priori analysis because of requirement of software and hardware for execution.	It is cheaper than Posteriori Analysis.
Maintenance Phase is required to tune the algorithm.	Maintenance Phase is not required to tune the algorithm.

3. Solve the following recurrence relation using substitution method. 5 Marks

T(n) = T(n) = 2 * T(n-1) + c1, (n > 1)

T(1) = 1

Ans: We know that the answer is probably T(N) = $O(2^n)$. The observation that we are almost doubling the number of O(1) operations for a constant decrease in n leads to the guess. We will thus show that T(n) <= k * 2^n - b and prove our answer.
Now we come over to the use of induction, here the base case occurs when n=1.

=> T(1) = 1 <= k * 21 - b = 2 * k - b.
=> 2 * k - b >= 1
=> k >= (b+1)/2

Inductive step: We assume that the property that we guessed holds for n - 1 and prove that in such a case it will also hold for n.

=> T(n) = 2 * (T(n-1)) + c1
=> T(n) <= 2 * (k * 2n-1 - b) + c1
=> T(n) <= k * 2n - 2 * b + c1
=> T(n) <= k * 2n - b (let, b = c1)

After the above steps we obtain, b = c1 and k = (b + 1) / 2. These equations have infinite solutions, and we are free to choose any constant as big-O notation doesn't care about constants being multiplied, divided, added or subtracted.
Hence, we have confirmed that $T(n) = O(2^n)$.

4. Solve the following recurrence relation using substitution method. 5 Marks

T(n) = 2 * T(n / 2) + n, n > 1

T(1) = 1

Ans: Base case (n = 1)

=> T(1) = 1 <= c1 * 1 * log(1) + c2
=> 1 < = c1 * 1 * 0 + c2
=> c2 >= 1

Inductive step, (n > 1)

=> T(n) <= 2 * T(n / 2) + n
=> T(n) <= 2 * (c1 * n / 2 * log(n / 2) + c2) + n
=> T(n) <= c1 * n * log(n) - c1 * n * log(2) + 2 * c2 + n
=> T(n) <= c1 * n * log(n) + (1 - c1 * log(2)) * n + 2 * c2

We can conclude that the above holds for some c1 >= 1 / log(2) and some c2. We again have numerous solutions for the above set of equations, and hence the guess is accurate and T(n) = O(n * log(n)).

5. Define Brute-Force method and discuss its advantages and disadvantages. 6 Marks

Definition: 2M Advantages: 2M Disadvantages: 2M

Ans: A brute force approach is an approach that finds all the possible solutions to find a satisfactory solution to a given problem. The brute force algorithm tries out all the possibilities till a satisfactory solution is not found.

Advantages

- This algorithm finds all the possible solutions, and it also guarantees that it finds the correct solution to a problem.
- This type of algorithm is applicable to a wide range of domains.
- It is mainly used for solving simpler and small problems.
- It can be considered a comparison benchmark to solve a simple problem and does not require any particular domain knowledge.

Disadvantages

- It is an inefficient algorithm as it requires solving each and every state.
- It is a very slow algorithm to find the correct solution as it solves each state without considering whether the solution is feasible or not.
- The brute force algorithm is neither constructive nor creative as compared to other algorithms.

6. Define and explain following asymptotic notations. 6Marks

Each for 2M

A. Big-Oh B. Big-omega C. Theta

Unit 2 QB

1. Solve the following Job sequencing with deadlines problem 6M

n=7,

Profits (p1, p2, p7) = {3, 5, 20, 18, 1, 6, 30}

Deadlines (d1, d2, d7) = {1, 3, 4, 3, 2, 1, 2}

Let n=7, Profits(p1, p2. ….p7)={3, 5, 20, 18, 1, 6, 30} Deadlines(d1, d2,….d7)={1, 3, 4, 3, 2, 1, 2} The feasible solution and their values are given below.

Sr. No	Feasible Solution	Frequenting Sequence	Value
1	(1,2)	1,2 or 2,1	8
2	(1,3)	3,1	23
3	(1,4)	4,1	21

Sr. No	Feasible Solution	Frequenting Sequence	Value
4	(1,5)	5,1	4
5	(1,6)	6,1	9
6	(1,7)	7,1	33
7	(2,3)	3,2	25
8	(3,4)	4,3	38
9	(4,5)	5,4	19
10	(5,6)	6,5	7
11	(6,7)	7,6	36
12	(1)	1	3
13	(2)	2	5
14	(3)	3	20

Sr. No	Feasible Solution	Frequenting Sequence	Value
15	(4)	4	18
16	(5)	5	1
17	(6)	6	6
18	(7)	7	30

Consider the jobs in the non-increasing order of profits subject to the constraint that the resulting job sequence J is a feasible solution.

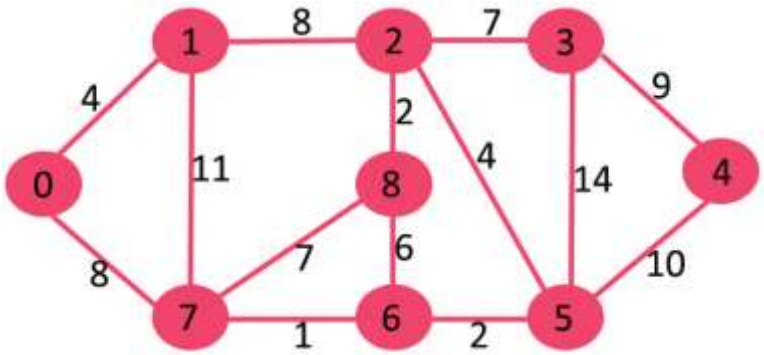
In the example considered before, the non-increasing profit vector is

30	**20**	**18**	**6**	**5**	**3**	**1**		**2**	**4**	**3**	**1**	**3**	**1**	**2**
p1	p2	p3	p4	p5	p6	p7		d1	d2	d3	d4	d5	d6	d7

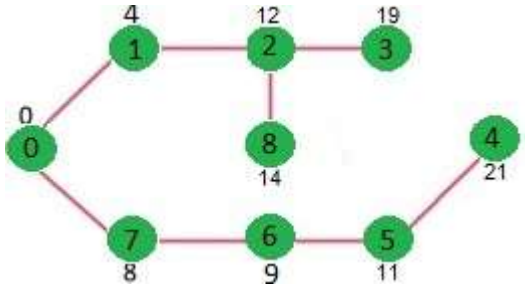
i. Now we start adding the job with J = 0 and Σ i ej Pi= 0.

ii. Job 7 is added to J as it has the largest profit and thusJ = {7} is a feasible one.

2. Find Dijkstra's shortest path from vertex 0 for the following graph 5Marks



Explanation: *The distance from 0 to 1 = 4.*
The minimum distance from 0 to 2 = 12. 0->1->2
The minimum distance from 0 to 3 = 19. 0->1->2->3
The minimum distance from 0 to 4 = 21. 0->7->6->5->4
The minimum distance from 0 to 5 = 11. 0->7->6->5
The minimum distance from 0 to 6 = 9. 0->7->6
The minimum distance from 0 to 7 = 8. 0->7
The minimum distance from 0 to 8 = 14. 0->1->2->8



3. **Find the optimal solution for the fractional knapsack problem making use of greedy approach.**

n = 5

w = 60 kg

(w1, w2, w3, w4, w5) = (5, 10, 15, 22, 25)

(b1, b2, b3, b4, b5) = (30, 40, 45, 77, 90)

Compute the value / weight ratio for each item-

Items	Weight	Value	Ratio
1	5	30	6
2	10	40	4
3	15	45	3
4	22	77	3.5
5	25	90	3.6

Sort all the items in decreasing order of their value / weight ratio-

I1	**I2**	**I5**	**I4**	**I3**
(6)	(4)	(3.6)	(3.5)	(3)

Start filling the knapsack by putting the items into it one by one.

Knapsack Weight	Items in Knapsack	Cost
60	Ø	0
55	I1	30
45	I1, I2	70
20	I1, I2, I5	160

Now,

- Knapsack weight left to be filled is 20 kg but item-4 has a weight of 22 kg.
- Since in fractional knapsack problem, even the fraction of any item can be taken.
- So, knapsack will contain the following items-

< I1 , I2 , I5 , (20/22) I4 >

Total cost of the knapsack

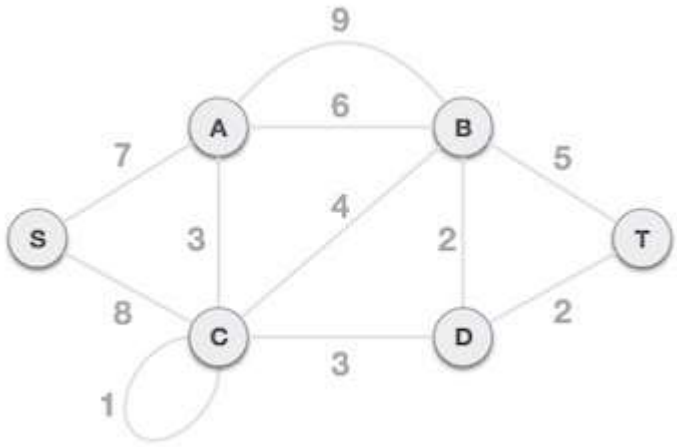
= 160 + (20/27) x 77

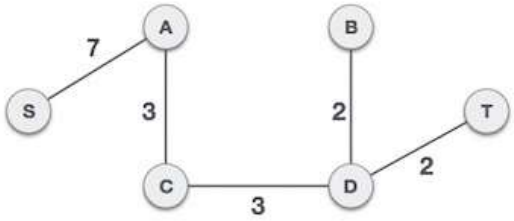
= 160 + 70

= 230 units

3. Write an algorithm to find the Minimum Spanning Tree using Kruskal algorithm and analyze it. 5 Marks

4. Construct the MST using Kruskal's algorithm for a graph given below. Also find minimum cost of it. 5 Marks





cost 17